UPGMA Phylogenetic Tree Construction Tool

Weronika Łaszkiewicz, 278434

June 8, 2025

1 Introduction

The Unweighted Pair Group Method with Arithmetic Mean (UPGMA) is a straightforward hierarchical clustering method for creating phylo-genetic trees (dendrograms). This project presents a complete Python application that constructs phylogenetic trees from biological data using the UPGMA method. The tool is designed to be flexible, accepting either a set of biological sequences or a pre-calculated distance matrix as input.

When sequences are provided, the program first performs a Multiple Sequence Alignment (MSA) using the Center-Star heuristic, calculates a Hamming distance matrix, and then proceeds with UPGMA clustering. The application is built with PyQt5 for a user-friendly graphical interface and utilizes the 'ete3' library for customizable tree visualization.

The complete source code and documentation are available in the project's GitHub repository:
github.com/wlaszkiewicz/phylogenetic-tree-upgma

2 System Design

2.1 Project Structure

The application follows a modular design, organized into a clear directory structure to separate concerns such as alignment logic, GUI components, and utilities.

```
UPGMA_Phylogenetic_Tool/
├── main.py ..... Main application entry point
├── alignment/
│   ├── center_star.py ..... Center-Star MSA and distance calculations
│   ├── needleman_wunsch.py ..... Pairwise global alignment
│   └── upgma.py ..... UPGMA clustering logic
├── gui/
│   ├── upgma_app.py ..... Main application window class
│   ├── add_sequence_dialog.py ..... Dialog for manual sequence input
│   ├── photo_viewer.py ..... Custom view for tree visualization
│   └── tree_renderer.py ..... Tree visualization rendering functions
└── utils/
    ├── fasta_parser.py ..... Utility for parsing FASTA format files
    └── file_saver.py ..... File saving operations for results export
```

- **main.py**: The main entry point script that initializes and runs the application.
- **alignment/**: Core bioinformatics algorithms package.
  - **center_star.py**: Implements:
    - Center-Star MSA algorithm
    - Hamming distance calculation
    - Ultrametric validation
    - Sum-of-pairs score calculation
  - **needleman_wunsch.py**: Optimal pairwise sequence alignment
  - **upgma.py**: Core UPGMA clustering logic
- **gui/**: GUI components package.
  - **upgma_app.py**: Main application class handling:
    - GUI layout and widgets
    - Event handling
    - Workflow coordination
  - **add_sequence_dialog.py**: Validated manual sequence input dialog
  - **photo_viewer.py**: Custom widget for tree visualization with zooming and panning functionalities
  - **tree_renderer.py**: Dedicated tree rendering functions that convert Newick string to visual tree, style it and handle temporary image.
- **utils/**: Helper utilities package.
  - **fasta_parser.py**: FASTA format parsing
  - **file_saver.py**: Results export functionality:
    - MSA data saving (FASTA/text)
    - Matrix data saving (CSV/text)
    - Tree image saving (PNG)
    - Newick format export

3 Algorithm Implementation

3.1 Mathematical Foundation

UPGMA is a hierarchical agglomerative clustering algorithm. It builds a rooted tree by iteratively merging the two closest clusters (nodes). The distance to the new, merged cluster from any other cluster is the arithmetic average of the distances from the original two clusters. A key assumption of UPGMA is the **molecular clock hypothesis**, which says that all lineages evolve at a constant rate. This assumption ensures the resulting tree is **ultrametric**, meaning the distance from the root to every leaf is identical.

The height of a new node formed by merging clusters i and j is half their distance:

$$\text{height}(i, j) = \frac{d(i, j)}{2} \tag{1}$$

The distance from a third cluster k to the new merged cluster (i, j) is:

$$d((i, j), k) = \frac{d(i, k) + d(j, k)}{2} \tag{2}$$

3.2 Input Validation and Ultrametricity

3.2.1 Input Requirements

- **Sequences**: Uppercase letters (A-Z) and gaps (-) only
- **Distance Matrix**:
  - Comma-separated names without spaces
  - Square numeric matrix (space/comma separated values)
  - Must be symmetric with zero diagonal

A critical step, especially when working with a user-provided distance matrix, is to validate its properties. The UPGMA algorithm's accuracy relies on the ultrametric property of the distance matrix. This tool implements a check for a strong version of this property known as the three-point condition.

For any three items (leaves) i, j, k, the three distances between them—$d(i, j)$, $d(j, k)$, and $d(i, k)$—must form an isosceles triangle where the two longest sides are equal.

The 'check_ultrametric_conditions' function iterates through all unique triplets of items, sorts their distances, and verifies that the two largest distances are approximately equal. If any violations are found, the application displays a warning dialog that informs the user and explains the potential impact on the reliability of the results, giving them the choice to continue with the calculation or terminate them.

3.3 Algorithm Overview

The program follows one of two workflows depending on the input:

1. **Sequence Input**:
   (a) Perform Multiple Sequence Alignment on the input sequences using the Center-Star method
   (b) Calculate a hamming distance matrix from the resulting alignment
   (c) Proceed with the UPGMA algorithm using the calculated matrix
2. **Matrix Input**:
   (a) Validate the user-provided distance matrix for correctness (square, symmetric, zero-diagonal)
   (b) Check if the matrix meets ultrametric conditions
   (c) Proceed directly with the UPGMA algorithm

3.3.1 Center-Star MSA

The Center-Star is used for distance calculation when sequences are provided. It has been **improved since the last task**, because I realized that it was unnecessarily complicated. This implementation, instead of aligning twice, stores the alignments that were used to get the center star and uses them later to insert gaps.

Algorithm 1 Center-Star MSA

Require: A set of sequences $S = \{s_1, s_2, ..., s_N\}$

Initialize an $N \times N$ matrix 'pairwise_scores'.
for all pairs of sequences (s_i, s_j) **do**
    Align s_i and s_j using Needleman-Wunsch algorithm.
    Store the score in 'pairwise_scores[i, j]' and 'pairwise_scores[j, i]'.
end for
Calculate the sum of scores for each row in 'pairwise_scores'.
Identify the sequence s_c with the highest total score as the "center" sequence.
Initialize a Multiple Sequence Alignment (MSA) with the center sequence s_c.
for all other sequences $s_i \in S$ where $i \neq c$ **do**
    Retrieve the pre-computed pairwise alignment of s_i and s_c.
    Add s_i to the MSA, inserting gaps into other rows as necessary to match the alignment of s_i with s_c.
end for
return The final MSA.

3.3.2 UPGMA Clustering

The core UPGMA process builds the tree from a distance matrix.

Algorithm 2 UPGMA Clustering

Require: Distance matrix 'D', list of item names 'L'

Initialize a list of clusters, where each item is its own cluster.
Initialize a dictionary of nodes where each item maps to a leaf node.
while number of clusters > 1 **do**
    Find the pair of clusters (C_i, C_j) with the minimum distance $d(C_i, C_j)$ in 'D'.
    Create a new parent node P for nodes corresponding to C_i and C_j.
    Set the height of the branches to the new node as $d(C_i, C_j)/2$.
    Create a new merged cluster $C_{new} = C_i \cup C_j$.
    Calculate distances from all other clusters C_k to C_{new} using the formula: $d(C_{new}, C_k) = \frac{|C_i|d(C_i,C_k)+|C_j|d(C_j,C_k)}{|C_i|+|C_j|}$.
    Update the distance matrix 'D' by removing rows/columns for C_i, C_j and adding one for C_{new}.
    Remove C_i, C_j from the list of clusters and add C_{new}.
end while
The final remaining cluster is the root of the tree.
Generate the Newick format string from the tree structure.

The algorithm is separated into a class UPGMACluster that represents each node in the hierarchy, whether it is a leaf or an ancestor. Each cluster object stores references to its left and right children, its calculated height in the tree, and the total number of leaves it subtends (leaf_count). This class also contains the recursive logic to generate the final Newick string output.

The main upgma_algorithm function manages the clustering process. It first initializes a leaf cluster for each input item and populates a dictionary with the initial distances. It then iteratively finds the closest pair of clusters, merges them into a new parent UPGMACluster, and updates the distance dictionary by calculating the weighted average distance to the new node. This process continues until only one root cluster remains, which represents the complete phylogenetic tree.

```
class UPGMACluster:

    def __init__(self, id_val, name=None, left=None, right=None, height=0.0):
        self.id = id_val
        self.name = name
        self.left = left
        self.right = right
        self.height = height
        self.leaf_count = 1 if not left else left.leaf_count + right.leaf_count

    def to_newick(self):
        if not self.left:
            return self.name
        return f"({self.left.to_newick()}:{self.height -
        ↪  self.left.height:.4f},{self.right.to_newick()}:{self.height - self.right.height:.4f})"

def upgma_algorithm(dist_matrix, item_names):
    n = len(item_names)
    if dist_matrix.shape[0] != n:
        return "ERROR: Matrix and names mismatch."
    if n == 0:
        return "();"
    if n == 1:
        return f"({item_names[0]}:0.0);"

    clusters = {i: UPGMACluster(id_val=i, name=name) for i, name in enumerate(item_names)}
    dists = {frozenset({i, j}): dist_matrix[i, j] for i in range(n) for j in range(i + 1, n)}
    next_cluster_id = n

    while len(clusters) > 1:
        if not dists:
            return "ERROR: Distance map became empty unexpectedly."

        min_pair_ids = min(dists, key=dists.get)
        id1, id2 = min_pair_ids
        c1 = clusters[id1]
        c2 = clusters[id2]
        new_height = dists[min_pair_ids] / 2.0
        new_cluster = UPGMACluster(next_cluster_id, left=c1, right=c2, height=new_height)

        new_dists = {}
        for other_id, other_cluster in clusters.items():
            if other_id not in min_pair_ids:
                d1 = dists[frozenset({id1, other_id})]
                d2 = dists[frozenset({id2, other_id})]
                new_dists[frozenset({next_cluster_id, other_id})] = (
                        (d1 * c1.leaf_count + d2 * c2.leaf_count) / (c1.leaf_count + c2.leaf_count)
                )

        del clusters[id1]
        del clusters[id2]
        dists = {k: v for k, v in dists.items() if id1 not in k and id2 not in k}
        clusters[next_cluster_id] = new_cluster
        dists.update(new_dists)
        next_cluster_id += 1

    return list(clusters.values())[0].to_newick() + ";"
```

3.4 Complexity Analysis

The computational complexity of the tool mostly depends on which workflow is used. The most demanding path is the one that starts from raw sequences, since it includes both Multiple Sequence Alignment (MSA) and UPGMA clustering. We'll assume N is the number of sequences and L is the length of the longest one.

Operation	**Time Complexity**	**Space Complexity**
Pairwise Alignments	$O(N^2 \cdot L^2)$	$O(N^2 \cdot L)$
Center-Star Build	$O(N \cdot L_{MSA})$	$O(N \cdot L_{MSA})$
UPGMA Clustering	$O(N^3)$	$O(N^2)$

Table 1: Computational complexity by primary operation.

- **Pairwise Alignments**: Every sequence has to be compared with every other one, which gives us $O(N^2)$ pairs. For each pair, we use the Needleman-Wunsch algorithm, which runs in $O(L^2)$. On top of that, we need space to store all the aligned sequences, which adds up to $O(N^2 \cdot L)$.
- **Center-Star Build**: Once all pairwise alignments are done, building the final multiple sequence alignment is relatively efficient. It just means going through each sequence and aligning it to the central profile. This step takes $O(N \cdot L_{MSA})$ time and space, where L_{MSA} is the length of the final alignment.
- **UPGMA Clustering**: The implementation uses a straightforward approach. For each of the $N-1$ steps, it searches through the current distance matrix to find the closest pair of clusters. In early steps, this involves scanning $O(N^2)$ entries, which leads to an overall time complexity of $O(N^3)$. Memory-wise, we need to store the full $O(N^2)$ distance matrix between clusters.

4 User Interface and Functionality

4.1 Interface Overview

The application provides a clean, tab-based interface with these key features:

- **Input Selection**: Radio buttons to switch between sequence or matrix input.
- **Data Entry**:
  - Sequences can be loaded from FASTA files or added manually via a dialog.
  - Distance matrix and names are entered in dedicated text fields.
- **Parameters**: Configurable match, mismatch, and gap scores for MSA.
- **Output Panel**: A multi-tab display showing all intermediate and final results.
- **Visualization**: An interactive panel for the graphical tree with zoom and pan controls.

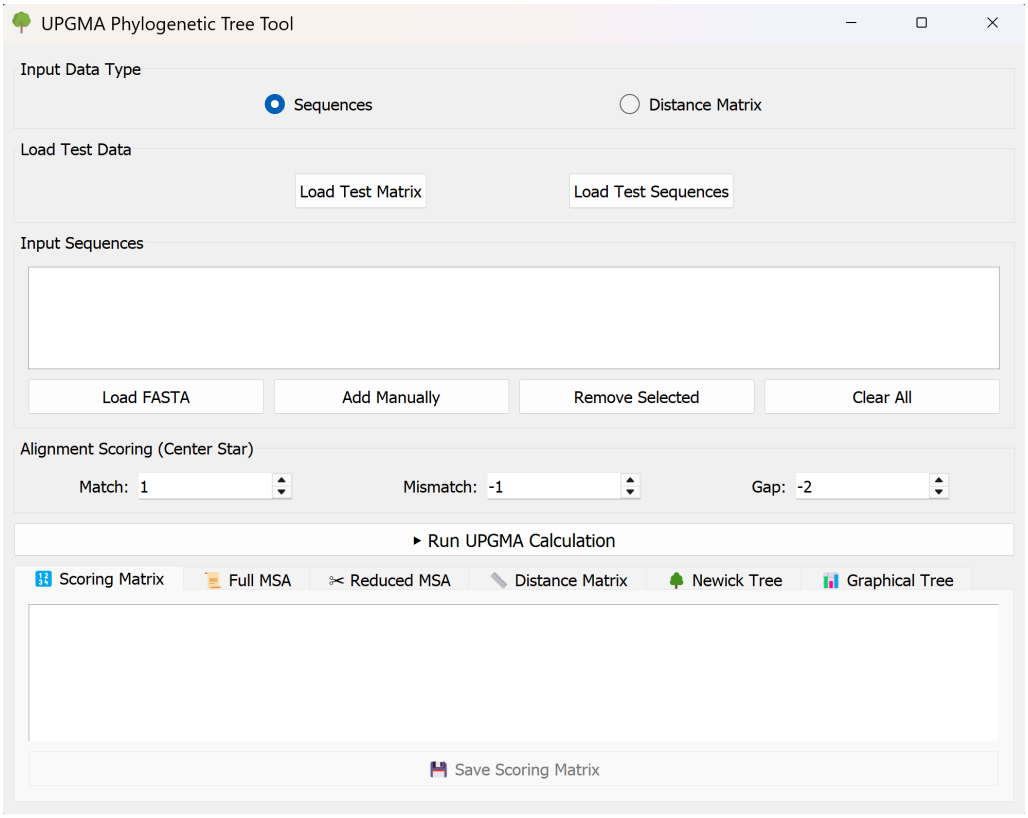

Figure 1: Main application window showing sequence input mode.

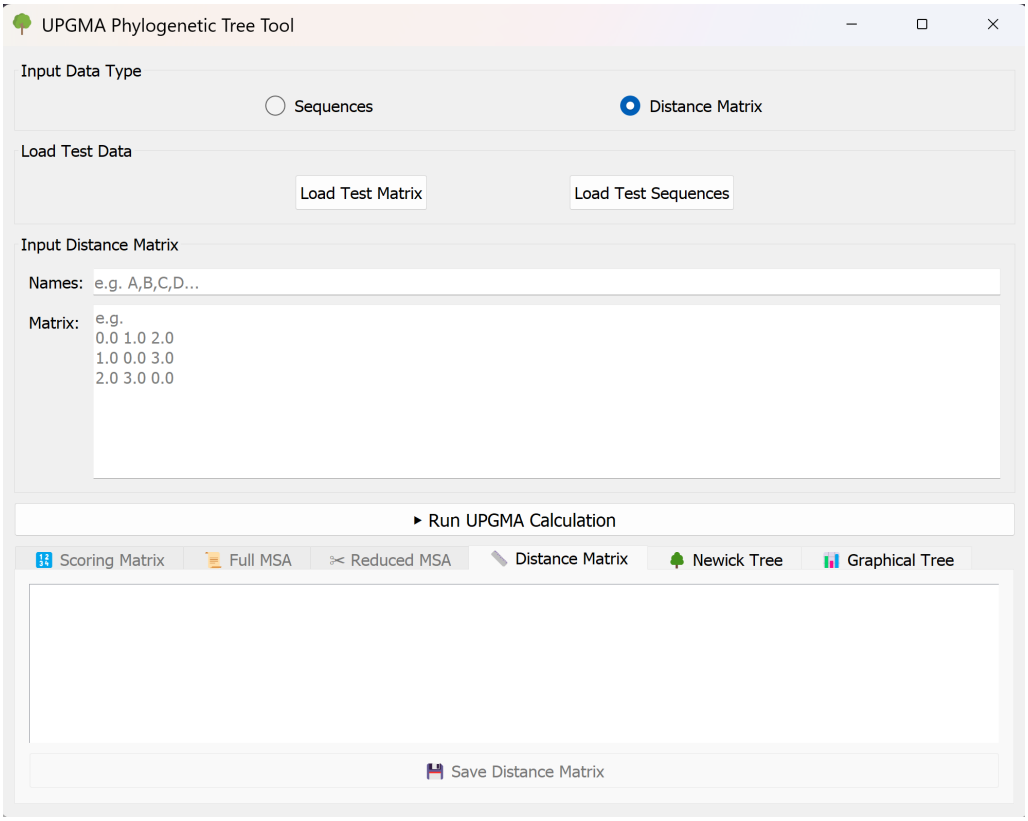

Figure 2: Main application window showing matrix input mode.

4.2 Output

The application's interface is dynamic and adapts to the selected input type. It only displays output tabs that are relevant to the current workflow.

- **When providing Sequences**:
  - Scoring Matrix, Full MSA, Reduced MSA, Distance Matrix, Newick Tree, and *Graphical Tree*.
- **When providing a Distance Matrix**: Only the relevant results are generated and displayed, and the rest of the tabs related to sequence alignment is disabled.
  - Distance Matrix, Newick Tree, and *Graphical Tree*.

5 User Manual

5.1 Basic Operation

1. **Choose Input Mode**: Select "Sequences" or "Distance Matrix".
2. **Provide Data**:
   - For sequences: Load from a FASTA file or add them one by one.
   - For matrix: Enter comma-separated names and the corresponding square distance matrix.
3. **Run Calculation**: Click the "Run UPGMA Calculation" button.
4. **View Results**: Navigate through the output tabs to inspect the scoring matrix, MSA, distance matrix, Newick string, and graphical tree.
5. **Save Results**: Use the "Save" button on any output tab to export the data.

5.2 Saving and Exporting Results

The application provides many options for saving all generated data. Each output tab has its own "Save" button with many format options:

- **Scoring and Distance Matrices**: Can be saved in two formats:
  - **CSV (*.csv)**: A standard comma-separated value file, easily importable into spreadsheets or data analysis tools.
  - **Plain Text (*.txt)**: A formatted text file with aligned columns for easy readability.
- **Full and Reduced MSA**: Can be saved in two formats:
  - **FASTA (*.fasta, *.fa)**: A standard bioinformatics format that includes sequence headers.
  - **Plain Text (*.txt)**: A simple text file showing the aligned sequences.

- **Newick Tree**: Can be saved as a standard **Newick file (*.nwk)**, which is compatible with most phylogenetic tree viewers.
- **Graphical Tree**: The rendered tree visualization can be saved as a high-quality **PNG image (*.png)**.

6 Example Analysis

Using the built-in test sequences:

```
s1_test   ATTGCCATT
s2_test   ATGGCCATT
s3_test   ATCCATTTTT
s4_test   ATCTTCTT
s5_test   ACTGACC
```

The application shows warning, because the matrix is not perfectly ultrametric.

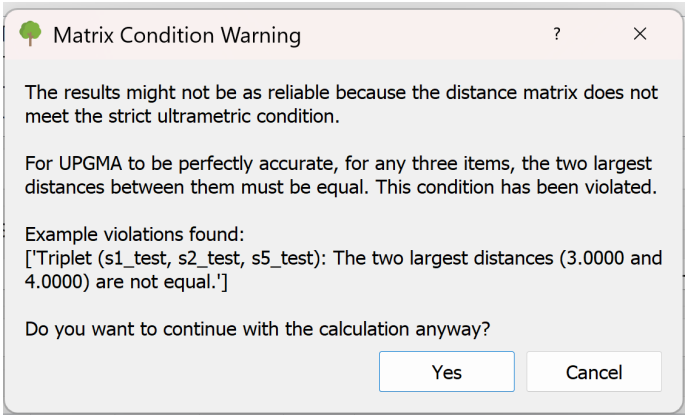

Figure 3: Warning dialog, asking the user if they want to terminate or continue.

6.1 Generated Results

The tool processes the input matrix and generates the pairwise scoring matrix, the full multiple sequence alignment (MSA), the reduced MSA, the distance matrix, and the final phylogenetic tree. In the graphical interface, each result is displayed within a scrollable tab. The images shown below **capture only the visible portions** of these outputs, the actual application allows users to **scroll and view** the entire contents of each matrix and table.

	s1_test	s2_test	s3_test	s4_test	s5_test	Sum
s1_test	0	7	-3	0	-3	**1**
s2_test	7	0	-3	0	-4	0
s3_test	-3	-3	0	2	-7	-11
s4_test	0	0	2	0	-3	-1

Figure 4: Pairwise Scoring Matrix

	1	2	3	4	5	6	7	8	9	10
s1_test	A	T	-	T	G	C	C	A	T	T
s2_test	A	T	-	G	G	C	C	A	T	T
s3_test	A	T	C	C	A	T	T	T	T	T
s4_test	A	T	-	C	T	T	C	-	T	T

(a) Full Multiple Alignment

	1	2	3	4	5	6
s1_test	A	T	T	G	T	T
s2_test	A	T	G	G	T	T
s3_test	A	T	C	A	T	T
s4_test	A	T	C	T	T	T

(b) Reduced Multiple Alignment

	s1_test	s2_test	s3_test	s4_test	s5_test
s1_test	0.0000	1.0000	2.0000	2.0000	3.0000
s2_test	1.0000	0.0000	2.0000	2.0000	4.0000
s3_test	2.0000	2.0000	0.0000	1.0000	5.0000
s4_test	2.0000	2.0000	1.0000	0.0000	5.0000

(c) Calculated Distance Matrix

Figure 5: The sequence alignment output.

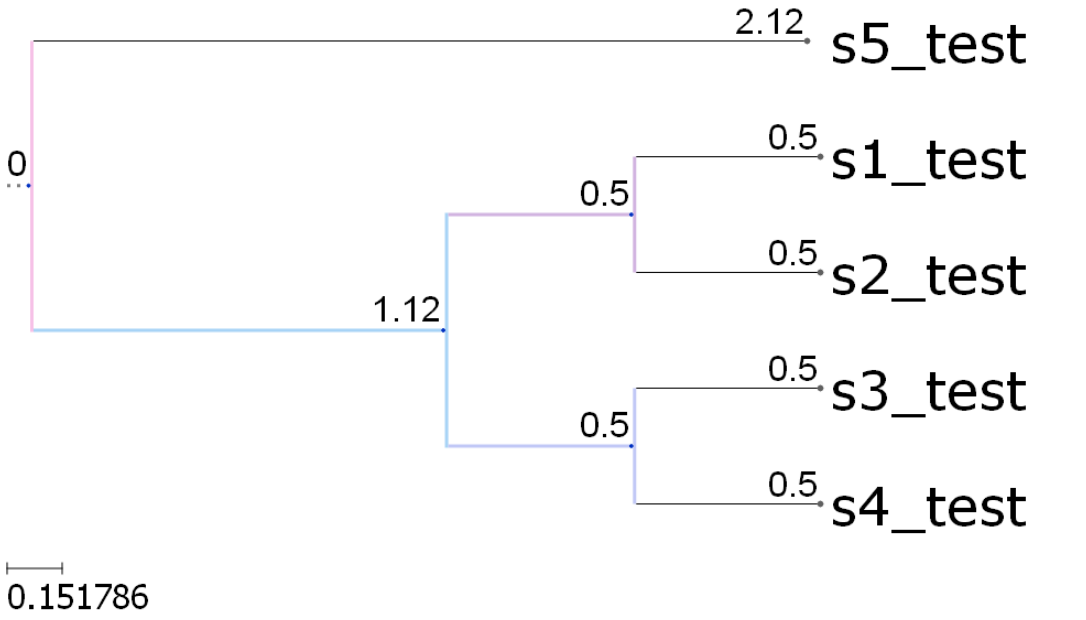

Figure 6: Final graphical tree rendered from the test sequences.

Newick string:

The corresponding Newick string provides a text-based representation of the tree structure and branch lengths.

```
(s5_test:2.1250,((s1_test:0.5000,s2_test:0.5000):0.5000,(s3_test
   :0.5000,s4_test:0.5000):0.5000):1.1250);
```

Additionally, all the results can be saved, for example in txt files:

Scoring matrix:

```
          s1_test   s2_test   s3_test   s4_test   s5_test   Sum
s1_test   0         7         -3        0         -3        1
s2_test   7         0         -3        0         -4        0
s3_test   -3        -3        0         2         -7        -11
s4_test   0         0         2         0         -3        -1
s5_test   -3        -4        -7        -3        0         -17
```

Aligments, full and reduced:

```
s1_test   AT-TGCCATT
s2_test   AT-GGCCATT
s3_test   ATCCATTTTT
s4_test   AT-CTTC-TT
s5_test   AC-TG--ACC
```

```
s1_test   ATTGTT
s2_test   ATGGTT
s3_test   ATCATT
s4_test   ATCTTT
s5_test   ACTGCC
```

Distance:

```
          s1_test   s2_test   s3_test   s4_test   s5_test
s1_test   0.0000    1.0000    2.0000    2.0000    3.0000
s2_test   1.0000    0.0000    2.0000    2.0000    4.0000
s3_test   2.0000    2.0000    0.0000    1.0000    5.0000
s4_test   2.0000    2.0000    1.0000    0.0000    5.0000
s5_test   3.0000    4.0000    5.0000    5.0000    0.0000
```

7 Conclusion

This project successfully implements a UPGMA phylogenetic tree construction tool that meets all the specified requirements. It offers dual input pathways, allowing users to start from sequences or a precomputed distance matrix. The tool performs thorough input validation, including a check for ultrametricity, and provides clear feedback to guide the user. From sequence alignment to final visualization, the entire workflow is integrated. The user-friendly and interactive graphical interface, developed with PyQt5, ensures accessibility and ease of use. Additionally, the project features tree rendering using the ete3 library. Users are provided with comprehensive output options that enable saving of all textual and graphical results generated by the tool.